A more honest theme contract.

Evidence study · strategy only · 22 October 2026

A sandboxed route to an extensible macOS appearance system; current inventory is only an audit snapshot.

41	27	14	419
existing records	mapped · partial	reference-only	v1 style rules

The key finding

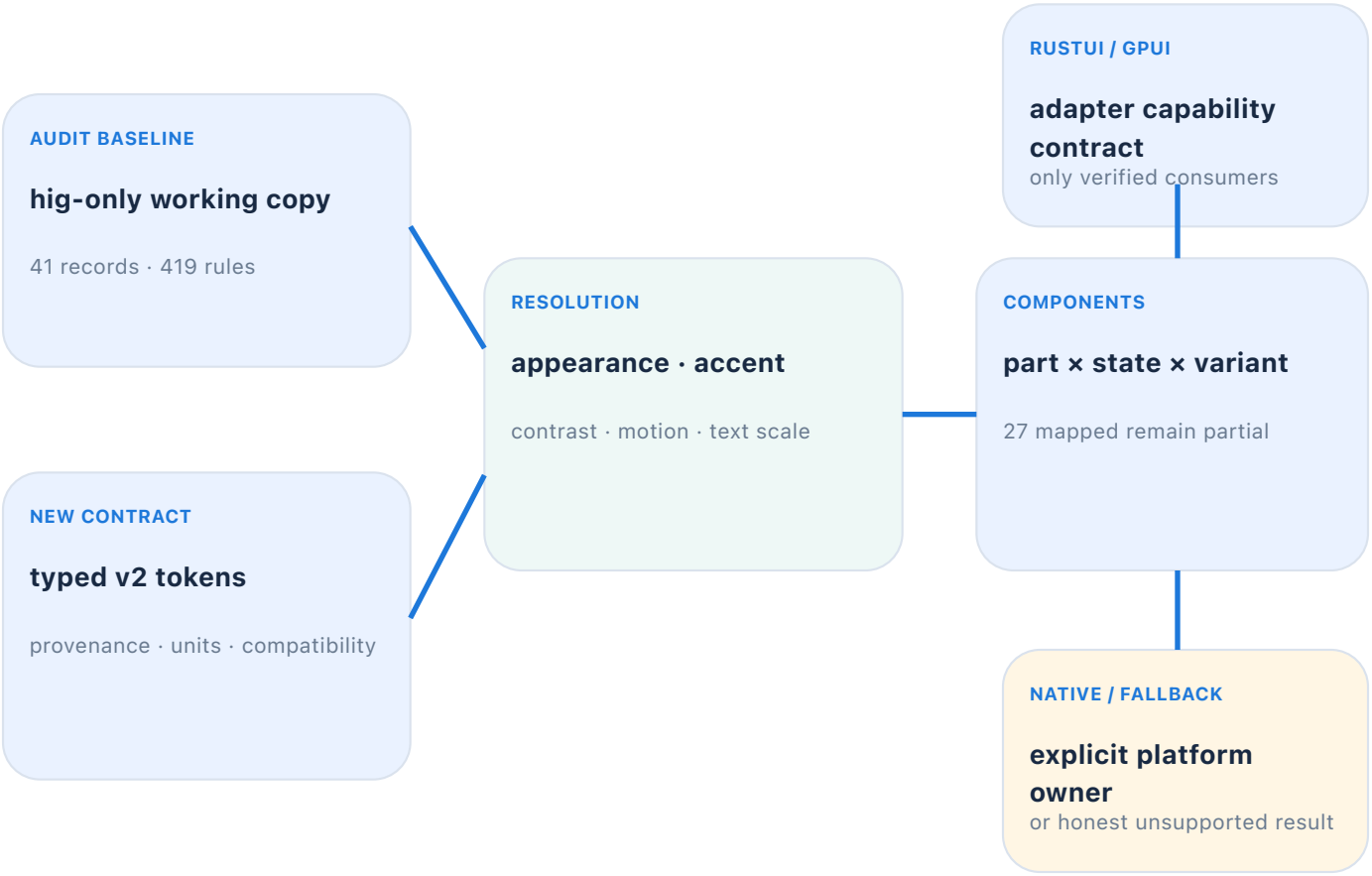
A registry mapping is not complete support. Actual renderer consumption is selective; native materials, accessibility settings, states and platform ownership need explicit runtime contracts and evidence.

ALL 41 RECORDS PRESERVED

NO IMPLEMENTATION OR TESTS

Proposed architecture: additive, typed, traceable

Preserve source data; make runtime ownership and resolved behavior explicit.



VALIDATE · ACCESSIBILITY · LOCALIZATION · VISUAL REGRESSION

The hig boundary matters

The study audited hig/macos27.json; the Playground embeds themes/macos27.json. Treat 41 rows as a baseline, not a ceiling. Expand the hig-only copy to cover missing components—even those absent from both JSON and code. Apply to rustui only after later approval.

What v2 must make explicit

The strategy separates appearance data from behavior the runtime must own.

Typed semantic tokens

Color, type, spacing, size, radius, stroke, icon, material, elevation and motion; explicit units, provenance and compatible references.

Deterministic resolution

Component override → state/variant → token → appearance → accessibility variant → system accent → platform range → named fallback.

Honest support claims

Capabilities are per renderer, part, property and state. Unsupported or uncovered values produce diagnostics; they are not silently treated as applied.

Platform ownership

RustUI/GPUI consume verified values; AppKit owns native panels, chrome and effects unless an explicit native adapter exists.

Complete behavior contract

Keyboard, focus, VoiceOver semantics, localization/RTL, dynamic text, resizing and reduced motion/transparency are runtime axes and tests.

Measured performance

Separate parse, validation, resolve, layout, repaint and backdrop costs; avoid per-frame JSON parsing and unsupported GPU claims.

The 41-row inventory is an audit snapshot, not a ceiling. Expand components in hig, including those absent from JSON and code; production integration needs later approval.

Roadmap · audit and catalogue expansion

PHASE A

A · Audit the hig boundary

Scope: Compare the two theme JSON sources and existing consumer evidence without editing RustUI.

Acceptance: 41 current records are mapped, 419 rules reconciled, and the source/runtime mismatch is explicit.

Gate: Every gap is labelled as source evidence, inference or unknown.

PHASE B

B · Expand the component inventory

Scope: Use screens, HIG evidence and current code to add missing widget families, parts, states and variants in hig only.

Acceptance: New concepts include provenance, platform ownership and known unsupported behavior. Existing 41 IDs remain traceable.

Gate: No 41-row cap; ambiguous boundaries and duplicated concepts are reviewed.

PHASE C

C · Draft the additive theme schema

Scope: Design typed semantic color, type, spacing, dimensions, icons, materials, motion and component contracts in hig.

Acceptance: Schema examples cover compatibility, references, units, diagnostics and legacy-data disposition without losing source values.

Gate: Draft stays in hig; no production schema/API edit.

PHASE D

D · Specify appearance resolution

Scope: Define system appearance, accent, contrast, text scale, reduced transparency/motion, fallbacks and deterministic precedence.

Acceptance: Decision tables show resolved value, owner, supported range, fallback and open questions.

Gate: Never claim native materials or system setting support without verified adapter evidence.

Roadmap · evidence, validation, approval

PHASE E

E · Build the component evidence matrix

Scope: Map every current and newly added family to semantic tokens, parts, states, interactions, screenshots and runtime evidence gaps.

Acceptance: Searchable matrix distinguishes concept, JSON declaration, registry mapping, consumer and tested behavior.

Gate: Mapped remains partial until each relevant property and state is proven.

PHASE F

F · Cover accessibility and adaptation

Scope: Specify focus/keyboard, VoiceOver semantics, dynamic text, contrast, localization/RTL, layout constraints and resize scenarios.

Acceptance: Each requirement has a testable scenario and known owner or explicit unresolved decision.

Gate: Mockups are scenarios, not proof; unsupported native behavior remains explicit.

PHASE G

G · Validate the hig working copy

Scope: Run static schema, consistency, provenance and coverage checks wholly inside hig when artifacts are ready.

Acceptance: All source records reconcile; new IDs and references are valid; reports name their input file and profile.

Gate: No runtime/build claims from static validation.

PHASE H

H · Prepare an integration decision

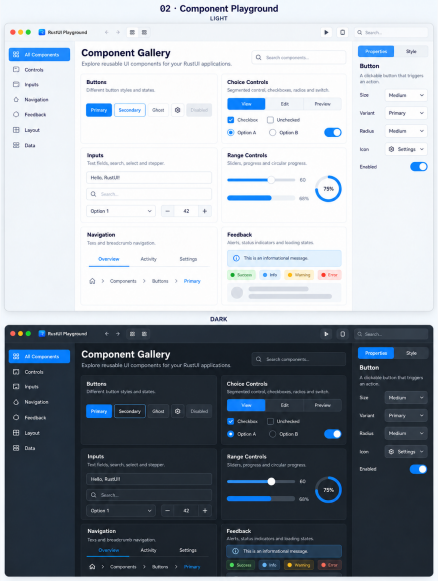
Scope: Summarize options, costs, API compatibility, unknowns, test needs and proposed sequence for the RustUI owner.

Acceptance: A concrete review package separates approved hig artifacts from proposed production changes.

Gate: Applying anything outside hig requires explicit later approval.

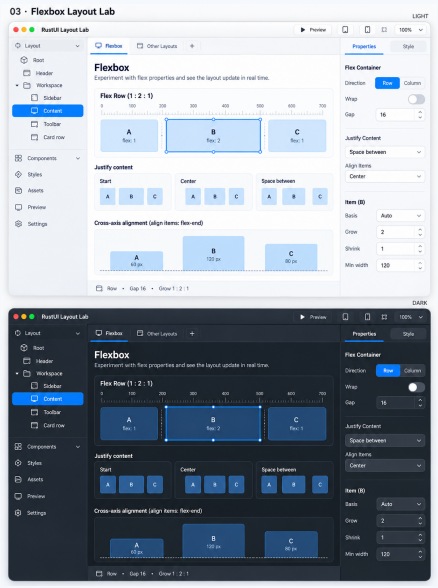
Before code: decide API scope, v1 deprecation, native AppKit scope, owners and measured budgets. Every phase requires separate approval.

Visual evidence · component and layout concepts



#02 · Component Playground

Light/dark component gallery concept; final gallery version. · Design concept, not proof of implementation or Apple specification.

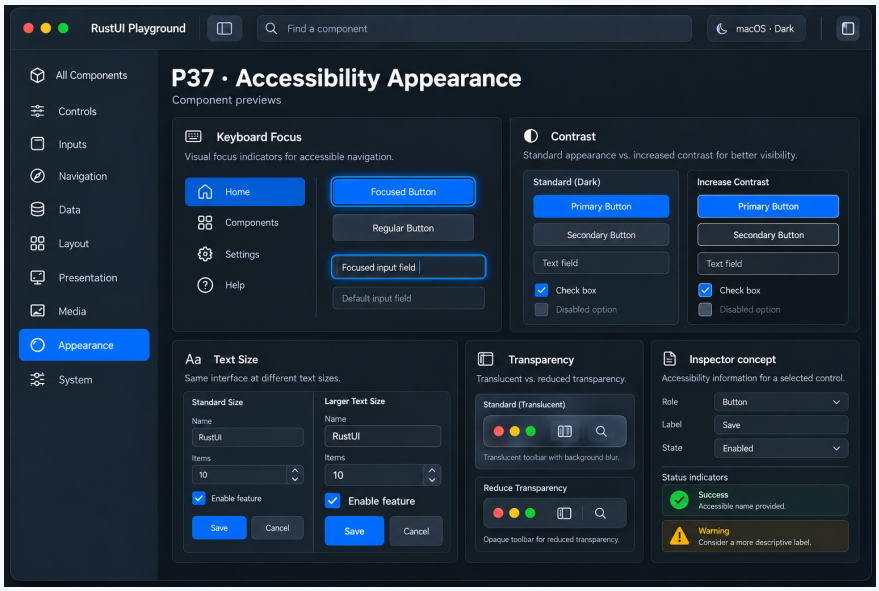


#03 · Flexbox Layout Lab

A concept for flex direction, grow/shrink, alignment and spacing. · Design concept, not proof of implementation or Apple specification.

Source correction: P05 Search/Tokens/Combo Boxes; P08 Menus/Pop-ups; P40 Desktop Widgets. The study map transposes/omits these labels.

Visual evidence · access and system concepts



P37 · Accessibility Appearance

Illustrative focus, contrast, text-size and transparency scenarios. · Design concept, not proof of implementation or Apple specification.



P40 · Desktop Widgets and System Controls

Illustrative system widgets and notification previews. · Design concept, not proof of implementation or Apple specification.

Source correction: P05 Search/Tokens/Combo Boxes; P08 Menus/Pop-ups; P40 Desktop Widgets. The study map transposes/omits these labels.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

01 link — 1 styles — MAPPED · PARTIAL

Boundary: Partial — mapped; foreground/typography capability.
Next: RustUI Link; semantic link + hover/focus/visited/disabled.

02 button — 2 styles — MAPPED · PARTIAL

Boundary: Partial — strongest v1 contract; typed metrics/roles, selector rules, GPUI palette and button resolver.
Next: RustUI action contract plus GPUI rendering; add variants, busy/destructive, focus/keyboard/a11y contract.

03 text — 4 styles — MAPPED · PARTIAL

Boundary: Partial — two opaque appearance parts mapped; vibrant parts not in the declared capability list.
Next: RustUI Text; role/style tokens and scale/wrap policy.

04 label — 4 styles — MAPPED · PARTIAL

Boundary: Partial — opaque light/dark text styles mapped; vibrant paths unclassified.
Next: RustUI Label; semantic label association separate from text appearance.

05 checkbox — 2 styles — MAPPED · PARTIAL

Boundary: Partial — background/foreground/typography style mapping; indeterminate/disabled semantics require behavior review.
Next: RustUI Checkbox; checked/mixed/disabled/focus/invalid and accessible value.

06 divider — 4 styles — MAPPED · PARTIAL

Boundary: Partial — two parts and background are mapped; exact direction/contrast behavior incomplete.
Next: RustUI decorative separator; orientation and semantic/hidden behavior.

07 icon — 9 styles — MAPPED · PARTIAL

Boundary: Partial — generic `icon` part width/height; other named icon parts are not declared renderer capabilities.
Next: RustUI Icon / platform symbol bridge; symbol source, rendering mode, RTL mirroring, accessibility.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

08 image — 4 styles — MAPPED · PARTIAL

Boundary: Partial — `image` width/height/radius; plural `images` artwork not mapped.
Next: RustUI Image; crop/fit/decorative vs informative labels, loading/error.

09 slider — 2 styles — MAPPED · PARTIAL

Boundary: Partial — size dimensions mapped; track/thumb semantics and keyboard/value feedback are not established by theme data.
Next: RustUI Slider; min/max/step/value/focus/increments and non-color value.

10 toggle — 2 styles — MAPPED · PARTIAL

Boundary: Partial — body bounds/radius/background mapped.
Next: RustUI Toggle; on/off/mixed/disabled, label/state and keyboard.

11 picker — 2 styles — MAPPED · PARTIAL

Boundary: Partial — body geometry/colors/typography mapped; option menu behavior is separate.
Next: RustUI Picker; popup/menu semantics, selected state and arrow/keyboard rules.

12 stepper — 10 styles — MAPPED · PARTIAL

Boundary: Partial — stepper buttons/separator/glyph paths mapped.
Next: RustUI Stepper; bounds, increments, min/max, repeat and accessibility value.

13 scroll_view — 2 styles — MAPPED · PARTIAL

Boundary: Partial — horizontal/vertical scrollbar background/border/radius capability; SVG metrics are used.
Next: RustUI scroll view; overlay visibility, track/thumb behavior, input, reduced motion.

14 text_field — 12 styles — MAPPED · PARTIAL

Boundary: Partial — body styles in small/large variants; `search`/control-specific artwork is not a complete input contract.
Next: RustUI TextField; caret/selection/IME/validation/placeholder/secure semantics.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

15 **search_field_examples** 2 styles REFERENCE-ONLY

Boundary: Reference-only — example composition, not a renderer.
Next: Preserve as reference; map tests to SearchField family, never mark record supported.

16 **radio_button** 2 styles MAPPED · PARTIAL

Boundary: Partial — geometry/color/typography mapped.
Next: RustUI Radio; grouping, selected/focus/keyboard, label, disabled.

17 **combo_box** 2 styles MAPPED · PARTIAL

Boundary: Partial — body geometry/colors/typography mapped.
Next: RustUI ComboBox; editable vs choice contract, popup and typeahead semantics.

18 **form_accessories** 6 styles REFERENCE-ONLY

Boundary: Reference-only — artwork composition, not a RustUI control.
Next: Preserve evidence; map desired accessory parts to IconButton/menu/disclosure contracts.

19 **alert_dialog** 4 styles REFERENCE-ONLY

Boundary: Reference-only — composition record; registry notes RustUI `AlertView` is inline.
Next: RustUI alert vs AppKit dialog owner decision; cancel/destructive/action order.

20 **save_dialog** 4 styles REFERENCE-ONLY

Boundary: Reference-only — system save-panel composition; no native panel consumer.
Next: Native AppKit integration gate; don't skin to imply system panel.

21 **menu** 2 styles MAPPED · PARTIAL

Boundary: Partial — background applied to GPUI popover; artwork filter/blend/vector are special-cased but reported as unresolved coverage.
Next: GPUI menu owner; menu roles, keyboard/shortcut, checked/mixed/submenu/disabled states.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

22 list — 2 styles — REFERENCE-ONLY

Boundary: Reference-only — record describes column header while RustUI `ListView` has no header part (registry reason).
Next: RustUI ListView separate contract; preserve record as non-applicable unless re-scoped.

23 notification — 2 styles — MAPPED · PARTIAL

Boundary: Partial — mapped body style properties; delivery/persistence/announcement semantics are outside theme.
Next: RustUI notification or OS notification integration decision; severity and actions.

24 window_chrome — 8 styles — REFERENCE-ONLY

Boundary: Reference-only — composition artwork; window chrome owned by host/window system.
Next: AppKit/GPUI native boundary; document unsupported until bridge exists.

25 toolbar — 0 styles — MAPPED · PARTIAL

Boundary: Partial via profile only — mapped name but zero style rules; toolbar height is read by toolbar renderer.
Next: RustUI toolbar; tokenize metrics, overflow/customization and accessibility behavior.

26 sidebar — 0 styles — MAPPED · PARTIAL

Boundary: Partial via GPUI profile roles only — mapped name but zero component rules.
Next: GPUI/RustUI navigation; active/inactive sidebar + source list selection contract.

27 text_style_guide — 265 styles — REFERENCE-ONLY

Boundary: Reference-only — typography source catalogue, not runtime component; profile exposes selected font sizes/styles separately.
Next: Reference data; derive typed typography scale and generated docs, avoid runtime record pretending to render.

28 utility_window — 2 styles — REFERENCE-ONLY

Boundary: Reference-only — window composition, no renderer mapping.
Next: Window-system/native owner gate.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

29 button_group — 6 styles — REFERENCE-ONLY

Boundary: Reference-only — composed control, not runtime component in registry.
Next: Consider grouped action contract after standalone buttons/segmented component.

30 color_picker — 6 styles — MAPPED · PARTIAL

Boundary: Partial — color wheel/swatches/crayons geometry/background capabilities.
Next: RustUI ColorPicker + native color panel distinction; gamut, value, keyboard and contrast.

31 disclosure_control — 4 styles — MAPPED · PARTIAL

Boundary: Partial — disclosure part foreground/typography capability.
Next: RustUI Disclosure; expanded/collapsed, arrow direction/RTL, keyboard and accessible state.

32 form — 4 styles — REFERENCE-ONLY

Boundary: Reference-only — source composition, no runtime form renderer record.
Next: RustUI composition contract; labels/errors/help/focus-order and responsive layout.

33 menu_bar — 0 styles — REFERENCE-ONLY

Boundary: Reference-only — no styles or RustUI menu-bar renderer.
Next: OS-owned menu bar; native command/action integration gate.

34 menu_item — 4 styles — MAPPED · PARTIAL

Boundary: Partial — foreground/typography capability.
Next: GPUI menu item; shortcut, checked/mixed, submenu, disabled, hover/focus and separator.

35 menu_separator — 4 styles — REFERENCE-ONLY

Boundary: Reference-only — record mismatch: popup separators owned by GPUI; source record names list/menu-bar separators.
Next: Keep reference-only; choose a separate semantic separator for menu if required.

Coverage appendix · component records (41 total)

Record classification only: registry-mapped records remain partial; reference-only records are retained source data.

36 **outline_view** 4 styles MAPPED · PARTIAL

Boundary: Partial — width/height/background part mapped.
Next: RustUI Tree/Outline; disclosure, selection, hierarchy, keyboard, row accessibility.

37 **search_field** 6 styles MAPPED · PARTIAL

Boundary: Partial — only `list_search` declared; toolbar/save-context parts unmapped.
Next: RustUI SearchField; query/clear/submit/suggestions/empty results and typeahead.

38 **segmented_control** 8 styles REFERENCE-ONLY

Boundary: Reference-only — no RustUI segmented-control renderer.
Next: New public component/API decision; preserve multiselect vs single-select distinction.

39 **tab_bar** 0 styles MAPPED · PARTIAL

Boundary: Partial via GPUI theme roles only — mapped name but no component style rules.
Next: RustUI TabView/GPUI; selection, overflow, close action, keyboard and badge roles.

40 **table** 4 styles MAPPED · PARTIAL

Boundary: Partial — table rows and headers have declared properties; header resolver special-cases vector artwork and coverage flags it unresolved.
Next: RustUI Table/DataGrid; sortable headers, selection, row/column navigation, density.

41 **window_controls** 8 styles REFERENCE-ONLY

Boundary: Reference-only — titlebar composition; no system window-control renderer.
Next: AppKit/window manager owns controls; native integration gate, never imitate as supported.

Evidence limits and decisions

Verified code: format v1 uses a strict envelope and partly opaque component payloads; selected profile fields map into GPUI roles; component consumers are selective. Presence in JSON alone does not prove use.

Working-copy boundary: hig/macos27.json was audited. The Playground embeds themes/macos27.json; the study did not run tests or regenerate a runtime coverage report against either file.

Unverified runtime: no build/test, macOS 27 display, AppKit material, VoiceOver/accessibility-tree, live reload, GPU or performance run. Four v1 records are empty: toolbar, sidebar, menu_bar and tab_bar.

Source gap: three early assistant turns in the Documents Search conversation and the initial user image payload were omitted in the available history. Missing material was not reconstructed.

Visual review: 86 unique originals had checksum/size verification and thumbnail contact-sheet inspection. Seven version-sensitive alternatives (#02, #16 and P28) got individual comparison; not every screen got full-size annotation.

Apple-source limit: official HIG titles/snippets and URLs were checked 2026-10-22; full JavaScript-rendered text was unavailable. These living pages are not a macOS 27 pixel spec.

Decisions before implementation

Public RustUI API or Playground profile? System or explicit appearance? v1 deprecation? RustUI/GPUI/AppKit ownership? OS accessibility scope? Baseline host and budgets? Full-size image annotation required?

Official Apple guidance · source register

Access checked 2026-10-22. Titles/short claims were searchable; full JavaScript-rendered page text was unavailable. These are living sources; recheck at design review.

01 **Designing for macOS**

macOS interaction context includes keyboard shortcuts and keyboard-only work styles; customization of colors/fonts appears in the page description. LIMIT: No numeric token values or exhaustive control-state table verified. https://developer.apple.com/design/human-interface-guidelines/designing-for-macos

02 **Color**

System-defined colors adapt to backgrounds, appearance, vibrancy and accessibility settings. Prefer semantic/system color roles and explicit accessible variants. LIMIT: Does not establish this theme's literal hex values as Apple values. https://developer.apple.com/design/human-interface-guidelines/color

03 **Materials**

Materials allow background color to pass through and help hierarchy/place recognition. LIMIT: Alpha/blur values in a JSON file do not create platform material composition. https://developer.apple.com/design/human-interface-guidelines/materials

04 **Typography**

Typography supports legibility, hierarchy and brand/style expression. LIMIT: Does not validate the catalogue's point sizes or guarantee scaling in RustUI. https://developer.apple.com/design/human-interface-guidelines/typography

05 **Accessibility**

Apple discusses contrast standards including WCAG and APCA; accessibility needs review beyond a color palette. LIMIT: A profile contrast threshold alone does not prove end-to-end accessibility. https://developer.apple.com/design/human-interface-guidelines/accessibility

06 **Dark Mode**

Dark Mode is a system appearance with a dark palette intended for low-light viewing. LIMIT: Current RustUI mode toggle is not evidence of following system appearance automatically. https://developer.apple.com/design/human-interface-guidelines/dark-mode

Official Apple guidance · source register

Access checked 2026-10-22. Titles/short claims were searchable; full JavaScript-rendered page text was unavailable. These are living sources; recheck at design review.

07 **Windows**

macOS windows have frame/body and can often be moved/resized by edges. LIMIT: Does not establish current RustUI window chrome/native ownership. https://developer.apple.com/design/human-interface-guidelines/windows

08 **Toolbars**

Toolbars group controls along an edge of a view. LIMIT: Customization, overflow and native integration require local implementation evidence. https://developer.apple.com/design/human-interface-guidelines/toolbars

09 **Sidebars**

Sidebar icons use accent by default; macOS system accent changes affect apps. LIMIT: RustUI does not currently prove system accent discovery from serialized alternatives. https://developer.apple.com/design/human-interface-guidelines/sidebars

10 **SF Symbols**

Multiple symbol rendering modes exist (monochrome, hierarchical, palette, multicolor). LIMIT: Availability/version/licensing and actual rendering remain a runtime/platform concern. https://developer.apple.com/design/human-interface-guidelines/sf-symbols

11 **Adopting Liquid Glass**

Apple's overview mentions preferred appearance and accessibility settings such as Reduce Transparency. LIMIT: Current RustUI adapter is not a native Liquid Glass implementation. https://developer.apple.com/documentation/technologyoverviews/adopting-liquid-glass

12 **AppKit `NSWorkspace.accessibilityDisplayShouldReduceTransparency`**

There is a native API for the macOS Reduce Transparency setting. LIMIT: The RustUI theme adapter shown does not call it. https://developer.apple.com/documentation/appkit/nsworkspace/accessibilitydisplayshouldreducetransparency